

Dokumentacja funkcjonalna - Wyznaczanie współrzędnych węzłów dla wizualnej reprezentacji grafu planarnego w języku C

 Aleksander Jóźwik, Anastasiya Kryvetskaya

 16 kwietnia 2026

Spis treści

1	Cel projektu	3
2	Funkcjonalności programu	3
3	Sposób wykonania programu	3
4	Lista dostępnych flag	4
5	Dane wejściowe	4
6	Komunikaty błędów	5
7	Dane wyjściowe	6
7.1	Plik tekstowy	6
7.2	Plik binarny	6
8	Sposób działania algorytmów	6
8.1	Algorytm sprężynowy Eadesa	6
8.2	Twierdzenie Tuttego	7
9	Schemat blokowy działania programu	9
10	Podział programu na moduły	10
11	Opis głównego pliku	11
12	Opis poszczególnych modułów	14
12.1	Moduł vector	14
12.1.1	Struktury	14
12.1.2	Funkcja add_vectors	14
12.1.3	Funkcja count_vector_length	14
12.1.4	Funkcja distance	15
12.1.5	Funkcja vector_from_points	15
12.1.6	Funkcja mult_by_num	15
12.1.7	Funkcja negative	16
12.1.8	Funkcja print_list	16
12.1.9	Funkcja subtract_vectors	17
12.2	Moduł validation	17

12.2.1	Funkcja is_graph_valid	17
12.3	Moduł output	17
12.3.1	Funkcja save_to_text	18
12.3.2	Funkcja save_to_bin	18
12.4	Moduł graph	19
12.4.1	Definicja struktur	19
12.4.2	Funkcja load_graph	20
12.4.3	Funkcja free_graph	20
12.4.4	Funkcja build_adj_list	21
12.4.5	Funkcja print_adj_list	21
12.4.6	Funkcja print_outer_face	22
12.4.7	Funkcja print_nodes_pos	22
12.4.8	Funkcja free_adj_list	23
12.4.9	Funkcja free_deg	23
12.4.10	Funkcja find_outer_face	23
12.4.11	Funkcja get_center	24
12.4.12	Funkcja place_on_circle	25
12.5	Moduł eades	25
12.5.1	Funkcja eades_algorithm	25
12.6	Moduł toutes	26
12.6.1	Funkcja toutes_algorithm	27
12.6.2	Funkcja tutte_iteration	29
12.6.3	Funkcja have_same_pos	30
12.6.4	Funkcja offset	30

1 Cel projektu

Celem projektu jest stworzenie programu w języku C, który służy do automatycznego i estetycznego rozmieszczania wierzchołków grafu planarnego w przestrzeni dwuwymiarowej, z wykorzystaniem algorytmu Eadesa lub twierdzenia Tuttego w zależności od wyboru użytkownika. Program działa w trybie nieinteraktywnym/wsadowym.

2 Funkcjonalności programu

Program oferuje następujące funkcjonalności:

1. Możliwość kontroli sposobu działania programu za pomocą argumentów linii poleceń.
2. Wczytywanie grafu z pliku tekstowego.
3. Rozmieszczanie wierzchołków za pomocą algorytmu Eadesa oraz twierdzenia Tuttego.
4. Zapisywanie informacji o położeniu wierzchołków po ich rozmieszczeniu do pliku tekstowego lub binarnego.

3 Sposób wykonania programu

Schemat wywołania programu wygląda w następujący sposób:

```
1 ./bin/main <plik_wejściowy> <flagi>
```

Na przykład:

```
1 ./bin/main examples/graph_1.txt -t b -o output.bin -s 1234 -a t
```

Powyższa komenda wywoła program w następujący sposób:

1. Program odczyta graf z pliku graph_1.txt.
2. Wynik programu zostanie zapisany do pliku binarnego (flaga -t).
3. Rezultat zostanie zapisany do pliku output.bin (flaga -o).
4. Program użyje do początkowego rozmieszczenia wierzchołków ziarna 1234 (flaga -s).
5. Program użyje twierdzenia Tuttego do rozmieszczenia wierzchołków (flaga -a).

4 Lista dostępnych flag

Nazwa flagi	Opis	Dostępne wartości	Domyślne wartości
-a	wybór algorytmu rozmieszczania wierzchołków	e, eades, t, tutte	e - Algorytm Eadesa
-t	typ pliku wyjściowego	t, txt, b, bin;	t - plik tekstowy
-o	nazwa pliku wyjściowego	-	output.txt lub output.bin
-s	ziarno używane dla początkowego rozmieszczenia wierzchołków	Nieujemna liczba całkowita	Program samodzielnie wybiera ziarno na podstawie czasu wykonania programu.
-w	szerokość obszaru	Dodatnia liczba całkowita.	100
-h	wysokość obszaru	Dodatnia liczba całkowita.	100
Parametry algorytmu Eadesa			
-m	Minimalna siła, poniżej której algorytm się zatrzymuje	Liczba zmiennoprzecinkowa	0.1
-i	Maksymalna liczba iteracji algorytmu	Liczba całkowita	10000
-l	Idealna długość krawędzi (sprężyny)	Liczba zmiennoprzecinkowa	10.0
-k	Stała sprężystości (siła przyciągania)	Liczba zmiennoprzecinkowa	1.0
-c	Stała odpychania	Liczba zmiennoprzecinkowa	2.0
-C	Współczynnik chłodzenia (spowalnia siły w czasie)	Liczba zmiennoprzecinkowa	0.97

Program nie wymaga podania żadnych flag do wykonania.

5 Dane wejściowe

Dane wejściowe mają następujący format:

```
1 <nazwa_krawędzi> <wierzchołek_A> <wierzchołek_B> <waga_krawędzi>^^I
```

Na przykład:

```
1 AB 1 2 1
2 BC 2 3 1
3 CD 3 4 1
```

Uwaga: Waga krawędzi nie jest używana przez algorytmy.

6 Komunikaty błędów

W przypadku napotkania błędu program zwraca następujące komunikaty.

Komunikat	Opis komunikatu	Kod powrotu
BŁĄD: Podano nieprawidłowy algorytm.	Wyświetlany, gdy wpisana wartość flagi -a jest różna od dostępnych. Program wykonuje się z algorytmem Eadesa.	-
BŁĄD: Podano nieprawidłowy typ pliku wyjściowego.	Wyświetlany, gdy wpisana wartość flagi -t jest różna od dostępnych. Program zapisuje do pliku tekstowego.	-
BŁĄD: Podano nieprawidłową szerokość/wysokość obszaru (wartość musi być większa od 0). Wybieram domyślną wartość = 100	Wyświetlany, jeżeli wpisana wysokość lub szerokość nie jest liczbą całkowitą lub nie jest > 0. Program wybiera domyślne wartości.	-
BŁĄD: Podano niepoprawne ziarno. Ziarno musi być nieujemne.	Wyświetlany, gdy podane ziarno jest mniejsze od 0. Program sam wybiera ziarno.	-
BŁĄD: Opcja -X wymaga podania wartości!	Wyświetlany, gdy nie podano wartości dla parametru, który wybrano. Program przerywa działanie.	1
BŁĄD: Nieznana opcja -X	Wyświetlany, gdy podano nieznany parametr. Program ignoruje parametr.	-
BŁĄD: Nie udało się otworzyć pliku.	Wyświetlany, gdy nie udało się otworzyć pliku o podanej nazwie. Program przerywa działanie.	2
BŁĄD: Nie udało się wczytać grafu.	Wyświetlany, gdy nie udało się wczytać grafu z otwartego pliku. Program przerywa działanie.	3
BŁĄD: Podany graf nie jest spójny.	Wyświetlany, gdy wczytany graf nie jest spójny. Program przerywa działanie.	4
BŁĄD: Podany graf nie jest planarny.	Wyświetlany, gdy wczytany graf nie jest planarny. Program przerywa działanie.	5
BŁĄD: Nie udało się zapisać do pliku tekstowego.	Wyświetlany, gdy nie udało się zapisać do pliku tekstowego o podanej nazwie. Program przerywa działanie.	6
BŁĄD: Nie udało się zapisać do pliku binarnego.	Wyświetlany, gdy nie udało się zapisać do pliku binarnego o podanej nazwie. Program przerywa działanie.	7
BŁĄD: Nie udało się zaalokować pamięci.	Wyświetlany, gdy nie udało się zaalokować pamięci.	8

7 Dane wyjściowe

Program oferuje możliwość zapisu do pliku tekstowego oraz binarnego.

7.1 Plik tekstowy

Dane w pliku tekstowym są zapisywane w następującym formacie:

```
1 <numer_wierzchołka> <współrzędna_x> <współrzędna_y>
```

Na przykład:

```
1 1 288.000000 288.000000
2 2 254.000000 254.000000
3 3 307.000000 307.000000
4 4 101.000000 101.000000
```

7.2 Plik binarny

Dane w pliku binarnym zapisywane są w następującej kolejności:

1. Liczba wierzchołków - `uint32_t` - 4 B
2. Identyfikator wierzchołka - `uint32_t` - 4 B
3. Współrzędna X wierzchołka - `float` - 4 B
4. Współrzędna Y wierzchołka - `float` - 4 B

Kroki 2, 3 i 4 powtarzane są dla każdego wierzchołka grafu. Zapis odbywa się zgodnie z metodą Little Endian - zapis od najmniej znaczącego bajtu.

8 Sposób działania algorytmów

8.1 Algorytm sprężynowy Eadesa

Ten algorytm liczy dla każdego wierzchołka sumę sił odpychania i przyciągania.

```
1 SpringEmbedder(G, minimalna_sila, maks_iteracji)
2 //minimalna_sila = bardzo mała wartość ogranicza działanie algorytmu i zatrzymuje pracę
3 //kiedy maksymalna znaleziona siła jest mniejsza niż minimalna_sila
4 t = 0
5 max_sila = duza_wartosc // maksymalna siła działająca na jeden z wierzchołków
6
7 while (t < maks_iteracji AND max_sila > minimalna_sila)
8
```

```

9   dla każdego wierzchołka v w G
10      oblicz siłę odpychania od wszystkich innych wierzchołków
11
12  dla każdej krawędzi (u, v) w G
13      oblicz siłę przyciągania między u i v
14
15  dla każdego wierzchołka v w G
16      zaktualizuj pozycję v na podstawie działającej siły
17      zaktualizuj max_sila
18
19  t = t + 1
20
21 end while

```

Formuła dla siły odpychania:

$$f_{\text{rep}}(p_u, p_v) = \frac{c_{\text{rep}}}{\|p_v - p_u\|^2} \cdot p_u \vec{p}_v \quad (1)$$

Gdzie:

- p_v, p_u – pozycje wierzchołków v i u
- c_{rep} – stała określająca siłę odpychania
- $p_u \vec{p}_v$ – wektor kierunkowy od u do v
- $\|\cdot\|$ – norma euklidesowa wektora

Formuła dla siły przyciągania:

$$f_{\text{attr}}(p_u, p_v) = c_{\text{spring}} \cdot \log\left(\frac{\|p_u - p_v\|}{\ell}\right) \cdot p_u \vec{p}_v \quad (2)$$

Gdzie:

- p_v, p_u – pozycje wierzchołków v i u
- c_{spring} – stała określająca siłę sprężyny
- ℓ – długość optymalna krawędzi
- $p_u \vec{p}_v$ – wektor kierunkowy od u do v
- $\|\cdot\|$ – norma euklidesowa wektora

8.2 Twierdzenie Tuttego

Ten algorytm działa szukając takiej pozycji dla każdego wierzchołka, aby jego współrzędne były średnią arytmetyczną pozycji jego sąsiadów.

```

1  TutteEmbedding(G)
2
3  wybierz wierzchołki zewnętrznej ściany grafu
4  ustaw ich pozycje na okręgu
5
6  dla każdego pozostałego wierzchołka v
7      zapisz równanie: pozycja v = średnia pozycji sąsiadów v
8
9  zbuduj układ równań liniowych dla współrzędnych x i y
10
11 rozwiąż układ równań
12
13 dla każdego wierzchołka v
14     ustaw jego pozycję według otrzymanego rozwiązania

```

Warunek barycentryczny:

$$p_v = \frac{1}{\deg(v)} \sum_{u \in N(v)} p_u \quad (3)$$

Gdzie:

- p_v – pozycja wierzchołka v na płaszczyźnie
- p_u – pozycja sąsiada u wierzchołka v
- $N(v)$ – zbiór sąsiadów wierzchołka v
- $\deg(v)$ – stopień wierzchołka v (liczba jego sąsiadów)
- $\sum$ – suma po wszystkich sąsiadach wierzchołka v

Równania liniowe dla współrzędnych:

$$x_v = \frac{1}{\deg(v)} \sum_{u \in N(v)} x_u \quad (4)$$

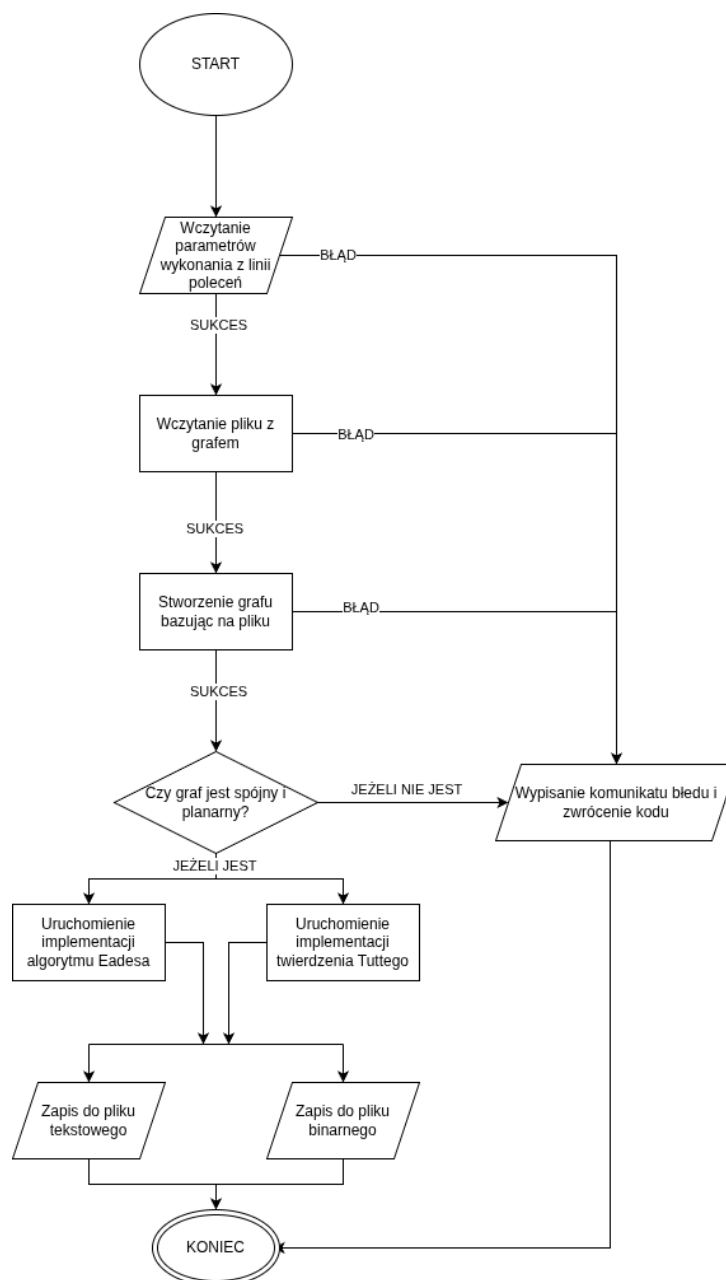
$$y_v = \frac{1}{\deg(v)} \sum_{u \in N(v)} y_u \quad (5)$$

Gdzie:

- x_v, y_v – współrzędne wierzchołka v
- x_u, y_u – współrzędne sąsiada u wierzchołka v
- $N(v)$ – zbiór sąsiadów wierzchołka v
- $\deg(v)$ – stopień wierzchołka v

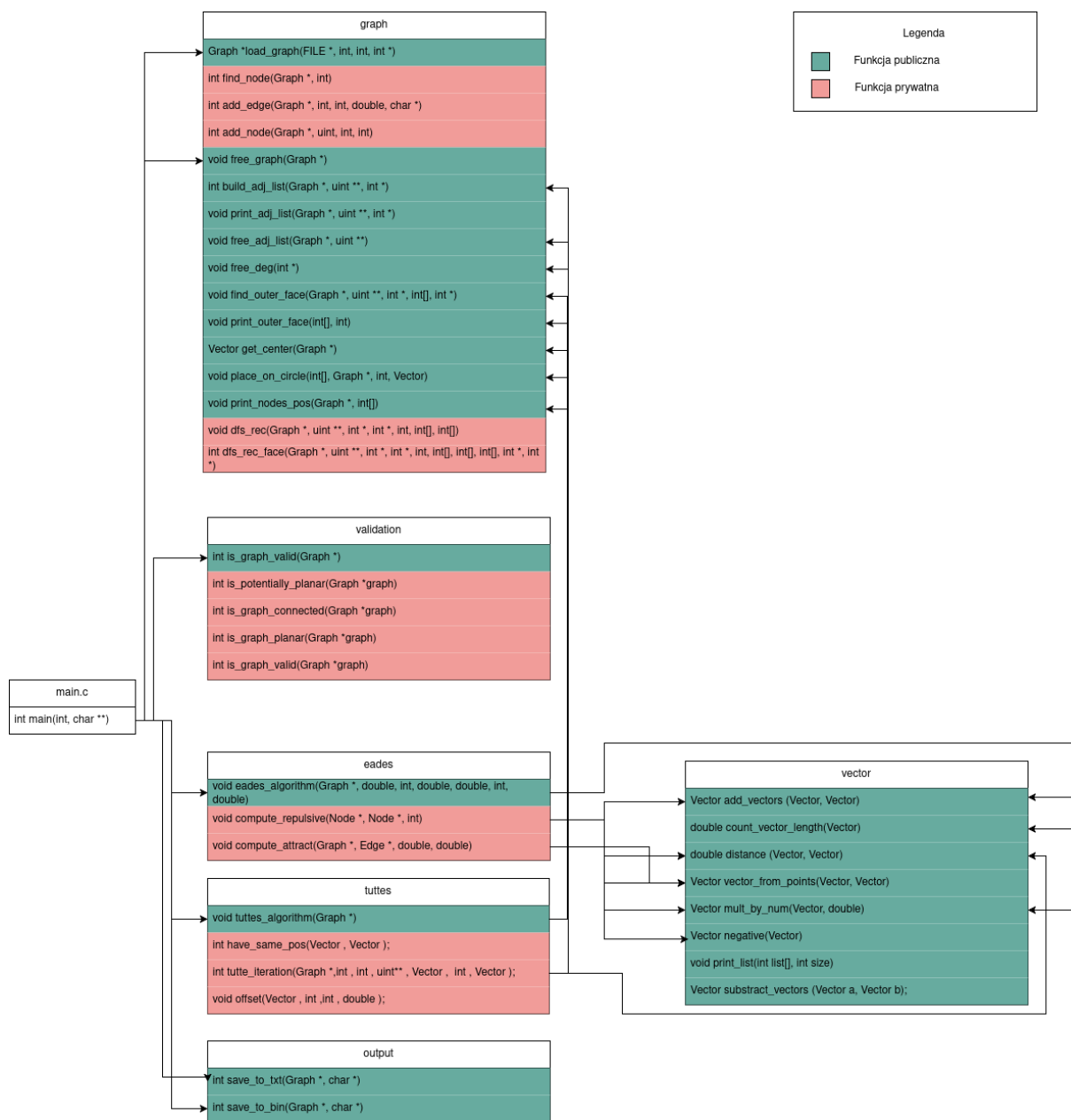
Spis treści

9 Schemat blokowy działania programu



Rysunek 1: Diagram blokowy ilustrujący sposób działania programu

10 Podział programu na moduły



Rysunek 2: Diagram podziału programu na pliki

Program został podzielony na 6 modułów z których każdy składa się z pliku C i pliku nagłówkowego. Moduły to:

- graph - odpowiada za zdefiniowanie struktur dla grafu, tworzenie grafu bazując na wczytanym pliku, zarządzanie pamięcią zaalokowaną dla struktur grafu.
- validation - odpowiada za sprawdzenie czy graf jest spójny i planarny.
- eades - odpowiada za uruchomienie funkcji implementującej algorytm Eadesa.
- tuttes - odpowiada za uruchomienie funkcji implementującej twierdzenie Tuttego.
- vector - odpowiada za wykonywanie operacji na wektorach.

- output - odpowiada za zapisywanie informacji o pozycji wierzchołków do pliku tekstowego lub binarnego.

11 Opis głównego pliku

Plik `main.c` zarządza przebiegiem programu oraz odpowiada za wczytywanie parametrów wykonania programu z linii poleceń.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <time.h>
6
7 #include "eades.h"
8 #include "graph.h"
9 #include "output.h"
10 #include "tuttes.h"
11 #include "error.h"
12 #include "validation.h"
```

Na początku program włącza pliki nagłówkowe.

```
1 int main(int argc, char **argv) {
```

Funkcja `main` przyjmuje ilość argumentów oraz tablicę argumentów z linii poleceń.

```
1 ^I// Zmienne dla parametrów wykonania
2     int algorithm = 0;
3     int output_type = OUTPUT_TXT;
4     char *output_name = NULL;
5     int opt;
6     FILE *graph_file = NULL;
7
8     int seed = 0;
9     int wasSeedProvided = 0;
10    int width = 100;
11    int height = 100;
12
13    // Parametry algorytmu Eadesa z wartościami domyślnymi
14    double minimum_force = 0.1;
15    int max_iterations = 10000;
16    double ideal_len = 10.0;
17    double repulsion_const = 2.0;
18    double spring_const = 1.0;
19    double cooling = 0.97;
```

Następnie inicjalizowane są zmienne potrzebne do pracy programu.

```
1 ^I// Wczytanie parametrów wykonania
2     while((opt = getopt(argc, argv, "a:t:s:o:w:h:m:i:l:k:c:C:")) != -1)
3     {
4         switch(opt)
5         {
6             // Wybór algorytmu
7             case 'a':
8                 if (strcmp(optarg, "t") == 0 || strcmp(optarg, "tutte") == 0) {
9                     algorithm = TUTTES_ALGORITHM; // 1 for Tutte
10                } else if (strcmp(optarg, "e") == 0 || strcmp(optarg, "eades") == 0) {
11                    algorithm = EADES_ALGORITHM; // 0 for Eades
12                } else {
13                    fprintf(stderr, "BŁĄD: Podano nieprawidłowy algorytm.\n");
```

```

14         algorithm = EADES_ALGORITHM;
15     }
16     break;
17     ^^I^^I^^I^^I
18     ^^I^^I^^I^^I[Kolejne flagi pominięto ...]
19
20     ^^I^^I^^I^^Icase ':':
21         fprintf(stderr, "BŁĄD: Opcja -%c wymaga podania wartości!\n", optopt);
22         return EXIT_FAILURE;
23         break;
24     case '?':
25         fprintf(stderr, "BŁĄD: Nieznana opcja -%c\n", optopt);
26         break;
27     }
28 }

```

W kolejnym kroku program korzysta z pętli while oraz funkcji getopt do wczytywania parametrów wykonania programu. Poprzez instrukcję switch funkcja sprawdza czy kolejne flagi i przyporządkowuje ich wartości do zmiennych. Pokazano jedynie kilka najważniejszych przypadków.

```

1  ^^I// Wczytanie pliku o nazwie wprowadzonej z linii poleceń
2  if (optind < argc) {
3      graph_file = fopen(argv[optind], "r");
4  }

```

Następnie main sprawdza czy podano dodatkowy parametr, którym jest nazwa pliku, jeżeli tak to plik jest zostaje wczytany.

```

1  ^^Iif (graph_file == NULL) {
2      fprintf(stderr, "BŁĄD: Nie udało się otworzyć pliku.\n");
3      return ERR_FILE_OPEN;
4  }

```

Program sprawdza czy plik został otworzony, jeśli nie to wyświetla komunikat o błędzie i zwraca kod.

```

1  ^^I// Sprawdzenie czy ziarno zostało podane jako argument linii poleceń
2  if (wasSeedProvided == 1)
3      srand(seed);
4  else
5      srand(time(NULL));

```

Jeżeli nie podano ziarna, program wybiera je samodzielnie, bazując na aktualnym czasie.

```

1  ^^I// Stworzenie grafu na podstawie pliku
2  int load_graph_status = 0;
3  Graph *graph = load_graph(graph_file, width, height, &load_graph_status);
4  if (load_graph_status == ERR_GRAPH_LOAD) {
5      fprintf(stderr, "BŁĄD: Nie udało się wczytać grafu.\n");
6      fclose(graph_file);
7      free(output_name);
8      return ERR_GRAPH_LOAD;
9  }
10 else if (load_graph_status == ERR_MEMORY_ALLOC) {
11     fprintf(stderr, "BŁĄD: Nie udało się wczytać grafu.\n");
12     fclose(graph_file);
13     free(output_name);
14     return ERR_MEMORY_ALLOC;
15 }

```

W tej części tworzony jest graf na podstawie otwartego pliku.

```

1  ^^Int validation_result = is_graph_valid(graph);
2  switch(validation_result) {
3      case ERR_GRAPH_NOT_CONNECTED:
4          fprintf(stderr, "BŁĄD: Graf nie jest spójny.\n");

```

```

5         return validation_result;
6     case ERR_GRAPH_NOT_PLANAR:
7         fprintf(stderr, "BŁĄD: Graf nie jest planarny.\n");
8         return validation_result;
9     case ERR_MEMORY_ALLOC:
10        fprintf(stderr, "BŁĄD: Błąd alokacji pamięci podczas walidacji grafu.\n");
11        return validation_result;
12    default:
13        break;
14    }

```

Następnie graf zostaje sprawdzony pod kątem spójności i planarności.

```

1  ^^I// Obsługa wyboru algorytmu z linii poleceń
2  switch(algorithm) {
3      case EADES_ALGORITHM:
4          eades_algorithm(graph, minimum_force, max_iterations, ideal_len, spring_const, repulsion_const,
5              ↪ cooling);
6          break;
7      case TUTTES_ALGORITHM:
8          tuttes_algorithm(graph);
9          break;
10     default:
11         eades_algorithm(graph, minimum_force, max_iterations, ideal_len, spring_const, repulsion_const,
12             ↪ cooling);
13         break;
14     }

```

Na podstawie wyboru użytkownika uruchomiony zostaje określony algorytm.

```

1  ^^I// Obsługa zapisu do wybranego typu pliku
2  switch(output_type) {
3      case OUTPUT_TXT:
4          if(save_to_txt(graph, output_name) == 1){
5              fprintf(stderr, "BŁĄD: Nie udało się zapisać do pliku tekstowego.\n");
6              return ERR_SAVE_TEXT_FAILED;
7          }
8          break;
9      case OUTPUT_BIN:
10         if(save_to_bin(graph, output_name) == 1){
11             fprintf(stderr, "BŁĄD: Nie udało się zapisać do pliku binarnego.\n");
12             return ERR_SAVE_BINARY_FAILED;
13         }
14         break;
15     }

```

W zależności od wybranego typu pliku program zapisuje dane wyjściowe.

```

1  ^^Ifree(output_name);
2  fclose(graph_file);
3
4  printf("Pomyślnie zapisano informacje o %d wierzchołkach do pliku.\n", graph->num_nodes);
5  free_graph(graph);
6
7  return EXIT_SUCCESS;
8  }

```

Na koniec program zwalnia pamięć, zamyka pliki i wyświetla komunikat o pomyślnym zapisaniu danych.

12 Opis poszczególnych modułów

12.1 Moduł vector

Moduł udostępnia funkcję służącą do wykonywania operacji na wektorach, a także definiuje ich strukturę. Funkcje tego modułu wykorzystywane są przez moduł eades i tuttes.

12.1.1 Struktury

```
1 typedef struct {  
2     double x;  
3     double y;  
4 } Vector;
```

Struktura wektora składa się dwóch liczb zmiennoprzecinkowych dla współrzędnych x i y.

12.1.2 Funkcja add_vectors

```
1 Vector add_vectors (Vector a, Vector b){  
2     a.x += b.x;  
3     a.y += b.y;  
4     return a;  
5 }
```

Funkcja dodaje do siebie dwa wektory.

Argumenty:

- a - pierwszy wektor
- b - drugi wektor

Funkcja zwraca zmieniony pierwszy wektor.

12.1.3 Funkcja count_vector_length

```
1 double count_vector_length(Vector a){  
2     return sqrt(a.x * a.x + a.y * a.y);  
3 }
```

Funkcja oblicza długość wektora.

Argumenty:

- a - wektor

Funkcja zwraca długość wektora jako wartość typu double.

12.1.4 Funkcja distance

```
1 double distance (Vector a, Vector b){
2     double dx = b.x - a.x;
3     double dy = b.y - a.y;
4     return sqrt(dx * dx + dy * dy);
5 }
```

Funkcja oblicza odległość między dwoma wektorami.

Argumenty:

- a - pierwszy wektor
- b - drugi wektor

Funkcja zwraca odległość między wektorami jako wartość typu double.

12.1.5 Funkcja vector_from_points

```
1 Vector vector_from_points(Vector a, Vector b){
2     Vector new;
3     new.x = b.x-a.x;
4     new.y = b.y-a.y;
5     return new;
6 }
```

Funkcja tworzy wektor przemieszczenia od punktu reprezentowanego przez wektor a do punktu reprezentowanego przez wektor b.

Argumenty:

- a - pierwszy wektor
- b - drugi wektor

Funkcja zwraca nowy wektor.

12.1.6 Funkcja mult_by_num

```
1 Vector mult_by_num(Vector a, double n){
2     a.x *=n;
3     a.y *=n;
4     return a;
5 }
```

Funkcja mnoży wektor przez skalar.

Argumenty:

- a - wektor
- n - wartość liczbowa typu double

Funkcja zwraca nowy wektor, będący wynikiem mnożenia wektora a przez skalar n.

12.1.7 Funkcja negative

```
1 Vector negative(Vector a){
2     Vector new;
3     new.x = -a.x;
4     new.y = -a.y;
5     return new;
6 }
```

Funkcja zmienia znak współrzędnych wektora.

Argumenty:

- a - wektor

Funkcja zwraca nowy wektor.

12.1.8 Funkcja print_list

```
1
2 void print_list(int list[], int size){
3     for (int i = 0; i < size; i++){
4         printf("[%d] - %d ", i, list[i]);
5     }
6     printf("\n");
7
8 }
9 }
```

Wypisuje zawartość tablicy liczb całkowitych na standardowe wyjście. Każdy element jest prezentowany wraz z odpowiadającym mu indeksem w formacie [indeks] - wartość.

Argumenty:

- list - tablica liczb całkowitych .
- size - liczba elementów znajdujących się w tablicy.

12.1.9 Funkcja subtract_vectors


```

1
2 Vector subtract_vectors (Vector a, Vector b){
3     a.x -= b.x;
4     a.y -= b.y;
5     return a;
6 }

```

Funkcja odejmowania dwóch wektorów dwuwymiarowych. Oblicza różnicę współrzędnych x oraz y.

Argumenty:

- a - wektor od którego odejmujemy.
- b - wektor który odejmujemy.

Funkcja zwraca nową strukturę Vector będącą wynikiem operacji a-b.

12.2 Moduł validation

Moduł ten odpowiada za sprawdzenie grafu pod kątem planarności i spójności po jego wczytaniu z pliku. Moduł składa się z 4 funkcji, które zostaną opisane poniżej.

12.2.1 Funkcja is_graph_valid

```

1 int is_graph_valid(Graph *graph){
2     if(is_potentially_planar(graph) == 0)
3         return ERR_GRAPH_NOT_PLANAR;
4     else if(is_graph_connected(graph) == 0){
5         return ERR_GRAPH_NOT_CONNECTED;
6     }
7     else if(is_graph_planar(graph) == 0){
8         return ERR_GRAPH_NOT_PLANAR;
9     }
10    return 0;
11 }

```

Jest to funkcja publiczna używana przez plik main.c, która używa innych funkcji tego modułu w celu weryfikacji grafu i w razie potrzeby zwrócenia odpowiednich kodów błędu.

Argumenty:

- graph - wskaźnik na strukturę grafu.

Funkcja zwraca liczbę całkowitą int informującą o rezultacie funkcji.

12.3 Moduł output

Moduł ten odpowiada za wypisywanie danych do pliku tekstowego lub binarnego.

12.3.1 Funkcja save_to_text

```
1 int save_to_text(Graph *graph, char *output_name) {
2     if(!graph) return 1;
3
4     char *final_name = output_name ? output_name : OUTPUT_NAME_TXT;
5
6     FILE *file = fopen(final_name, "w");
7     if(!file) return 1;
8
9     for(int i = 0; i < graph->num_nodes; i++) {
10         fprintf(file, "%d %lf %lf\n", graph->nodes[i].id, graph->nodes[i].position.x,
11             ↪ graph->nodes[i].position.y);
12     }
13     fclose(file);
14     return 0;
15 }
```

Funkcja ta odpowiada za wypisywanie informacji o wierzchołkach grafu do pliku tekstowego o wybranej nazwie. Jeżeli nazwa nie została podana to zostaje ona ustawiona na domyślną ("output.txt"). Plik tekstowy jest zgodny z formatem ustalony w dokumentacji funkcjonalnej.

```
1 <numer_wierzchołka> <współrzędna_x> <współrzędna_y>
```

Argumenty:

- graph - wskaźnika na strukturę grafu
- output_name - nazwa pliku wyjściowego

Funkcja zwraca liczbę całkowitą równą 1 jeżeli operacja się niepowiodła i 0 jeżeli się powiodła.

12.3.2 Funkcja save_to_bin

```
1 int save_to_bin(Graph *graph, char *output_name) {
2     if(!graph) return 1;
3
4     char *final_name = output_name ? output_name : OUTPUT_NAME_BIN;
5
6     FILE *file = fopen(final_name, "wb");
7     if(!file) return 1;
8
9     uint32_t num_nodes = (uint32_t) graph->num_nodes;
10    fwrite(&num_nodes, sizeof(uint32_t), 1, file);
11    for(int i = 0; i < graph->num_nodes; i++) {
12        uint32_t index = (uint32_t) graph->nodes[i].id;
13        float x = (float) graph->nodes[i].position.x;
14        float y = (float) graph->nodes[i].position.y;
15
16        fwrite(&index, sizeof(uint32_t), 1, file);
17        fwrite(&x, sizeof(float), 1, file);
18        fwrite(&y, sizeof(float), 1, file);
19    }
20    fclose(file);
21    return 0;
22 }
```

Funkcja `save_to_bin` służy do wypisywania danych do pliku binarnego. Podobnie jak dla poprzedniej funkcji, jeżeli nazwa pliku nie została podana to jest ona ustawiona na domyślną ("output.bin"). Dane są zapisywane w kolejności i typie zdefiniowanym w dokumentacji funkcjonalnej.

Argumenty:

- `graph` - wskaźnika na strukturę grafu
- `output_name` - nazwa pliku wyjściowego

Funkcja zwraca liczbę całkowitą równą 1 jeżeli operacja się niepowiodła i 0 jeżeli się powiodła.

12.4 Moduł graph

Moduł ten służy do tworzenia i zarządzania grafami.

12.4.1 Definicja struktur

```
1 typedef struct {
2     int u, v;
3     double weight;
4     char *name;
5 } Edge;
6
7 typedef struct {
8     Vector position;
9     Vector force;
10    uint id;
11 } Node;
```

Zdefiniowane są struktury dla wierzchołków oraz krawędzi grafu. Należy zwrócić szczególną uwagę na fakt, że pole `id` w wierzchołku nie jest używane w dalszej części programu oprócz w czasie wypisywania do pliku. Jest ono traktowane tylko jako etykieta. Aby operować na wierzchołkach program stosuje indeks tabeli `nodes` struktury grafu.

```
1 typedef struct{
2     Node *nodes;
3     int num_nodes;
4     int capacity_nodes;
5
6     Edge *edges;
7     int num_edges;
8     int capacity_edges;
9     int width;
10    int height;
11 } Graph;
```

Zdefiniowano również strukturę grafu. Która zawiera tablice dynamiczne wierzchołków i krawędzi wraz z ich ilością elementów oraz pojemnością. Struktura zawiera również szerokość i wysokość obszaru, w którym wyświetlony będzie graf.

12.4.2 Funkcja `load_graph`

Funkcja ta tworzy graf na podstawie wczytanego pliku.

Argumenty:

- graph_file - wskaźnik na plik
- width - szerokość obszaru, w którym wyświetlony będzie graf
- height - wysokość obszaru, w którym wyświetlony będzie graf
- out_code - kod błędu

Funkcja zwraca wskaźnik na strukturę grafu.

```
1 graph load_graph(plik, szerokość, wysokość, out_kod):
2     1. JEŚLI plik nie istnieje -> ustaw błąd i zwróć NULL.
3
4
5     2. ALOKUJ pamięć dla struktury Graph:
6         - Ustaw początkową pojemność (16) dla wierzchołków i krawędzi.
7         - Zainicjalizuj wymiary obszaru wyświetlania.
8         - JEŚLI błąd alokacji -> zwróć NULL.
9
10    3. DOPÓKI można odczytać dane z pliku (identyfikator, u, v, waga):
11        - Szukaj wierzchołka 'u' w grafie:
12        - Jeśli nie istnieje -> dodaj go do tablicy.
13        - Szukaj wierzchołka 'v' w grafie:
14        - Jeśli nie istnieje -> dodaj go do tablicy.
15        - DODAJ krawędź między 'u' a 'v' z podaną wagą i nazwą.
16
17        - JEŚLI podczas dodawania zabraknie pamięci -> zwolnij wszystko i zwróć NULL.
18
19    4. ZWRÓĆ gotowy wskaźnik na graf.
```

12.4.3 Funkcja free_graph

Funkcja służy do zwalniania pamięci zaalokowanej dla struktury grafu.

Argumenty:

- graph - wskaźnik na graf

```
1 void free_graph(Graph *graph) {
2     if (graph) {
3         for (int i = 0; i < graph->num_edges; i++) {
4             free(graph->edges[i].name);
5         }
6
7         free(graph->edges);
8         free(graph->nodes);
9         free(graph);
10    }
11 }
```

Funkcja dealokuje nazwy krawędzi oraz tablice krawędzi i wierzchołków oraz samą strukturę grafu. Oto fragment dokumentacji implementacyjnej przygotowany na podstawie dostarczonego kodu, z zachowaniem Twojej składni LaTeX oraz oryginalnych komentarzy.

12.4.4 Funkcja build_adj_list

```
1 int build_adj_list(Graph *graph, uint **adj_list, int *deg) {
2     int n_nodes = graph->num_nodes;
3     int n_edges = graph->num_edges;
4
5     // Zapełnienie deg
6     for (int i = 0; i < n_nodes; i++)
7         deg[i] = 0;
8
9     // Zapełnienie adj_list
10    for (int i = 0; i < n_edges; i++) {
11        int u = graph->edges[i].u;
12        int v = graph->edges[i].v;
13
14        adj_list[u][deg[u]++] = v;
15        adj_list[v][deg[v]++] = u;
16    }
17    return EXIT_SUCCESS;
18 }
```

Funkcja ta przekształca listową reprezentację krawędzi w listę sąsiedztwa, co jest używane przez algorytm tuttesa dla efektywnego poszukiwania sąsiadów. W pierwszym kroku zeruje tablicę stopni wierzchołków, a następnie iteruje po krawędziach, dodając sąsiadów do odpowiednich list.

Argumenty:

- graph - wskaźnik na strukturę grafu.
- adj_list - tablica dwuwymiarowa przechowująca sąsiadów wierzchołków.
- deg - tablica przechowująca liczbę sąsiadów dla każdego wierzchołka.

Funkcja zwraca EXIT_SUCCESS po poprawnym zbudowaniu struktury.

12.4.5 Funkcja print_adj_list

(*)Funkcja służy wyłącznie dla dyagnostyki. Jej wywołanie jest zakomentowane w kodzie.

```
1 void print_adj_list(Graph *graph, uint **adj_list, int *deg) {
2     for (int u = 0; u < graph->num_nodes; u++) {
3         printf("Sąsiedzi dla %d: ", u);
4         for (int i = 0; i < deg[u]; i++)
5             printf("%d ", adj_list[u][i]);
6         printf("\n");
7     }
8 }
```

Wypisuje kompletną listę sąsiedztwa na standardowe wyjście. **Argumenty:**

- graph - wskaźnik na strukturę grafu.
- adj_list - tablica przechowująca sąsiadów.
- deg - tablica stopni wierzchołków.

Przykład wyjścia:

```

1 Sąsiedzi dla 0: 1
2 Sąsiedzi dla 1: 0 2 3
3 Sąsiedzi dla 2: 1 3
4 Sąsiedzi dla 3: 2 1

```

12.4.6 Funkcja print_outer_face

(*)Funkcja służy wyłącznie dla dyagnostyki. Jej wywołanie jest zakomentowane w kodzie.

```

1 void print_outer_face(int dfs_res[], int dfs_res_size) {
2     for (int i = 0; i < dfs_res_size; i++) {
3         printf("%d", dfs_res[i]);
4         printf("->");
5     }
6     printf("%d", dfs_res[0]);
7     printf("\n");
8 }

```

Wizualizuje cykl tworzący zewnętrzny poligon grafu (outer face) w formacie czytelnej ścieżki. Pierwszy i ostatni element zawsze będą takie same, ponieważ jest to prezentacja zamkniętego cyklu

Argumenty:

- dfs_res - tablica przechowująca indeksy wierzchołków poligonu zewnętrznego.
- dfs_res_size - liczba elementów w tablicy.

Przykład wyjścia:

```

1 3->2->1->3

```

12.4.7 Funkcja print_nodes_pos

(*)Funkcja służy wyłącznie dla dyagnostyki. Jej wywołanie jest zakomentowane w kodzie.

```

1 void print_nodes_pos(Graph *graph, int is_fixed[]) {
2     for (int i = 0; i < graph->num_nodes; i++) {
3         printf("[%d] - (%.3f,%.3f) ", i, graph->nodes[i].position.x,
4             graph->nodes[i].position.y);
5         if (is_fixed[i] == 1) {
6             printf("-poligon zewnętrzny");
7         }
8         printf("\n");
9     }
10    printf("=====");
11    printf("\n");
12 }

```

Wypisuje współrzędne wszystkich wierzchołków, oznaczając te, które zostały przypisane do poligonu zewnętrznego.

Argumenty:

- graph - wskaźnik na strukturę grafu.

- `is_fixed` - tablica flag logicznych (1 - wierzchołek stały(z poligonu zewnętrznego), 0 - ruchomy).

Przykład wyjścia:

```
1 [0] - (30.000,72.000)
2 [1] - (55.000,48.000) -poligon zewnetrzny
3 [2] - (100.000,72.000) -poligon zewnetrzny
4 [3] - (36.000,28.000) -poligon zewnetrzny
5 =====
```

12.4.8 Funkcja `free_adj_list`

Funkcja służy do zwalniania pamięci zaalokowanej dla struktury listy sąsiedztwa.

```
1 void free_adj_list(Graph *graph, uint **adj_list) {
2     for (int i = 0; i < graph->num_nodes; i++)
3         free(adj_list[i]);
4     free(adj_list);
5 }
```

Argumenty:

- `graph` - wskaźnik na strukturę grafu (używany do określenia liczby wierzchołków).
- `adj_list` - tablica sąsiadów do usunięcia.

12.4.9 Funkcja `free_deg`

Funkcja służy do zwalniania pamięci zaalokowanej dla struktury ilości sąsiadów.

```
1 void free_deg(int *deg) {
2     if (!deg)
3         return;
4     free(deg);
5 }
```

Argumenty:

- `deg` - wskaźnik na tablicę stopni wierzchołków.

12.4.10 Funkcja `find_outer_face`

```
1 void find_outer_face(Graph *graph, uint **adj_list, int *deg, int cycle_res[],
2 int *cycle_idx) {
3     // Inicjalizacja narzędzi
4     int n = graph->num_nodes;
5     int visited[n];
6     int parent[n];
```

```

7  int found = 0;
8  int start = 0;
9
10 // Zapełniamy zerami visited[], czyli "nic nie zwiedzono"; -1 w parent[]
11 // znaczy że ten wierzchołek nie ma rodzica
12 for (int i = 0; i < n; i++) {
13     visited[i] = 0;
14     parent[i] = -1;
15 }
16 *cycle_idx = 0;
17
18 dfs_rec_face(graph, adj_list, deg, &start, 0, visited, parent, cycle_res,
19             cycle_idx, &found);
20
21 }

```

Funkcja ta stanowi punkt wejścia do algorytmu wyznaczania zewnętrznego poligonu. Jej zadaniem jest przygotowanie struktur pomocniczych (tablicy odwiedzin oraz tablicy przodków) i uruchomienie dedykowanego algorytmu przeszukiwania w głąb (dfs_rec_face), który zidentyfikuje pierwszy napotkany cykl.

Argumenty:

- graph - wskaźnik na strukturę grafu.
- adj_list - dynamiczna tablica dwuwymiarowa (lista sąsiedztwa).
- deg - tablica przechowująca stopnie wierzchołków.
- cycle_res - tablica, do której zostaną zapisane indeksy wierzchołków tworzących cykl.
- cycle_idx - wskaźnik na zmienną przechowującą liczbę znalezionych wierzchołków w cyklu.

12.4.11 Funkcja get_center

```

1 Vector get_center(Graph *graph) {
2     Vector center;
3     center.x = round((graph->width) / 2);
4     center.y = round((graph->height) / 2);
5     return center;
6 }

```

Funkcja oblicza współrzędne środka obszaru, na którym rysowany jest graf. Wynik jest wyznaczany na podstawie wymiarów (width i height) zapisanych w strukturze grafu.

Argumenty:

- graph - wskaźnik na strukturę grafu zawierającą wymiary obszaru roboczego.

Funkcja zwraca strukturę Vector zawierającą współrzędne punktu środkowego.

12.4.12 Funkcja place_on_circle


```

1 void place_on_circle(int outer_faces[], Graph *graph, int k, Vector center) {
2     // margin to odległość krańca okręga od granicy przestrzeni
3     int margin = 20;
4     // radius- promień domyślnego okręga na którym rozkładamy wierzchołki
5     double radius = fmin(graph->width, graph->height) / 2.0 - margin;
6
7     // Iteracja po wierzchołkach zewnętrznego poligonu
8     for (int i = 0; i < k; i++) {
9
10        // Kąt pomiędzy kolejnymi wierzchołkami
11        double angle = 2.0 * M_PI * i / k;
12
13        // Zmiana pozycji wierzchołków
14        graph->nodes[outer_faces[i]].position.x =
15        center.x + radius * cos(angle);
16        graph->nodes[outer_faces[i]].position.y =
17        center.y + radius * sin(angle);
18    }
19 }

```

Funkcja odpowiada za geometryczne rozmieszczenie wierzchołków wyznaczonych jako „poligon zewnętrzny” na okręgu. Jest to kluczowy etap przygotowawczy dla algorytmu Tutte’a, ponieważ unieruchomienie tych wierzchołków w formie wypukłego wielokąta gwarantuje poprawne rozmieszczenie pozostałych punktów wewnątrz.

Argumenty:

- outer_faces - tablica indeksów wierzchołków do rozmieszczenia.
- graph - wskaźnik na strukturę grafu, w której zostaną zaktualizowane pozycje.
- k - liczba wierzchołków w wielokącie zewnętrznym.
- center - wektor współrzędnych środka, wokół którego budowany jest okrąg.

12.5 Moduł eades

Moduł ten zawiera implementację algorytmu Eadesa, należącego do grupy algorytmów siłowych (*force-directed graph drawing*). Służy on do automatycznego rozmieszczania wierzchołków grafu w przestrzeni dwuwymiarowej tak, aby krawędzie miały zbliżoną długość, a wierzchołki nie nakładały się na siebie.

12.5.1 Funkcja eades_algorithm

```

1 void eades_algorithm(Graph *graph,
2     double minimum_force, int max_iterations,
3     double ideal_len, double spring_const, int c, double cooling)
4 {
5     int t = 0;
6     double max_force_magnitude = 1.0;
7     double temperature = 1.0;
8
9     while(t < max_iterations && max_force_magnitude > minimum_force) {
10         max_force_magnitude = 0.0;
11
12         // Wyzeruj siły dla bieżącej iteracji
13         for(int i = 0; i < graph->num_nodes; i++) {

```

```

14     graph->nodes[i].force.x = 0;
15     graph->nodes[i].force.y = 0;
16 }
17
18 // Oblicz siły odpychania między wszystkimi parami wierzchołków
19 for(int i = 0; i < graph->num_nodes; i++) {
20     for(int j = i + 1; j < graph->num_nodes; j++) {
21         compute_repulsive(&graph->nodes[i], &graph->nodes[j], c);
22     }
23 }
24
25 // Oblicz siły przyciągania wzdłuż krawędzi
26 for(int i = 0; i < graph->num_edges; i++) {
27     compute_attract(graph, &graph->edges[i], spring_const, ideal_len);
28 }
29
30 // Zaktualizuj pozycje wierzchołków i znajdź maksymalną siłę
31 for(int i = 0; i < graph->num_nodes; i++) {
32     Node *node = &graph->nodes[i];
33     double current_force_magnitude = count_vector_length(node->force);
34
35     if(current_force_magnitude > max_force_magnitude) {
36         max_force_magnitude = current_force_magnitude;
37     }
38
39     // Zastosuj siłę do pozycji wierzchołka, skalując przez temperaturę
40     Vector displacement = mult_by_num(node->force, temperature);
41     node->position = add_vectors(node->position, displacement);
42 }
43
44 // Schładzanie
45 temperature *= cooling;
46 t++;
47 }
48 }

```

Główna funkcja sterująca procesem układania grafu. W każdej iteracji obliczane są siły odpychania (po między wszystkimi wierzchołkami) oraz siły przyciągania (tylko wzdłuż krawędzi). Pozycje wierzchołków są aktualizowane o wektor wypadkowej siły pomnożony przez aktualną „temperaturę” układu, która maleje w czasie, co pozwala na stabilizację rysunku.

Argumenty:

- graph - wskaźnik na strukturę grafu.
- minimum_force - próg siły, poniżej którego algorytm kończy pracę.
- max_iterations - limit iteracji pętli.
- ideal_len - docelowa długość krawędzi.
- spring_const - stała sprężystości (przyciąganie).
- c - stała potrzebna dla obliczenia siły odpychania.
- cooling - współczynnik chłodzenia układu (z zakresu 0 do 1).

12.6 Moduł tuttes

Moduł ten zawiera implementację algorytmu Tutte'a (metoda barycentryczna), służącego do wyznaczania planarnego ułożenia grafu. Algorytm opiera się na unieruchomieniu wierzchołków poligonu zewnętrznego na planie wielokąta wypukłego i rozmieszczeniu pozostałych wierzchołków w środkach ciężkości ich sąsiadów.

12.6.1 Funkcja `tuttes_algorithm`

Jest to główna funkcja sterująca procesem rozmieszczania grafu. Proces ten został podzielony na fazę przygotowawczą oraz iteracyjną pętlę obliczeniową.

Argumenty:

- `graph` - wskaźnik na strukturę grafu.
- `max_iterations` - maksymalna liczba kroków algorytmu.

Funkcja zwraca `EXIT_SUCCESS` w przypadku powodzenia lub kod błędu alokacji.

```
1 int tuttes_algorithm(Graph *graph, int max_iterations) {
2     //Inicjalizacja potrzebnych narzędzi i alokacja
3     int n = graph->num_nodes;
4     int idx = 0;
5     int dfs_res[n];
6     uint **adj_list = (uint **)malloc(n * sizeof(uint *));
7     int *deg = (int *)malloc(n * sizeof(int));
8     Vector new_pos [n];
9
10
11     for(int i = 0; i < n; i++)
12         new_pos[i] = graph->nodes[i].position;
13     for (int i = 0; i < n; i++){
14         adj_list[i] = (uint *)malloc(n * sizeof(uint));
15     }
16
17     double max_change;
18     double difference = 0.08 * (graph->width + graph->height) / 2.0;
19
20     // Sprawdzenie działania alokacji
21     if (adj_list == NULL) {
22         fprintf(stderr, "Błąd: Nie udało się zaalokować pamięci.\n");
23         return ERR_MEMORY_ALLOC;
24     }
25     if (deg == NULL) {
26         fprintf(stderr, "Błąd: Nie udało się zaalokować pamięci.\n");
27         return ERR_MEMORY_ALLOC;
28     }
29 }
```

W pierwszej części funkcja alokuje pamięć na listy sąsiedztwa oraz pomocniczą tablicę `new_pos`, która przechowuje nowo obliczone współrzędne przed ich ostatecznym zapisaniem do struktury grafu.

```
1 // Budowanie listy sąsiedztwa
2 int result_adj_list = build_adj_list(graph, adj_list, deg);
3 if (result_adj_list != 0) {
4     fprintf(stderr, "Błąd: Nie udało się zbudować listy sąsiedztwa.\n");
5     return ERR_ADJ_LIST;
6 }
7 // Wypisywanie listy sąsiedztwa
8 //print_adj_list(graph, adj_list, deg);
9
10 // 2.Znalezienie dowolnego cyklu wierzchołków grafu
11 find_outer_face(graph, adj_list, deg, dfs_res, &idx);
12
13 // Wypisywanie poligonu zewnętrznego
14 //print_outer_face(dfs_res, idx);
15
16
17 //3.=====Część obliczeniowa=====
18 // Przygotowanie tablicy wierzchołków poligonu zewnętrznego (fixed)
19 int is_fixed[n];
20 for (int i = 0; i < n; i++) {
21     is_fixed[i] = 0;
22 }
```

```

22     }
23     for (int i = 0; i < idx; i++) {
24         is_fixed[dfs_res[i]] = 1;
25     }
26     // Ustawienie środka i rozmieszczenie punktów zewnętrznych na okręgu
27     Vector center = get_center(graph);
28     place_on_circle(dfs_res, graph, idx, center);
29

```

Następnie graf jest analizowany w celu znalezienia cyklu (ściany zewnętrznej). Wierzchołki należące do tego cyklu są oznaczane jako fixed (stałe) i rozmieszczane na okręgu za pomocą funkcji `place_on_circle`.

```

1 //Wypisywanie pozycji wierzchołków po rozstawianiu wierzchołków poligonu zewnętrznego
2 //print_nodes_pos(graph, is_fixed);
3
4 int t = 0;
5 max_change = 10.0;
6 /// 4. Główna pętla obliczeniowa
7 while(t < max_iterations && max_change > MINIMUM_CHANGE) {
8
9     t++;
10    max_change = 0.0;
11
12    //4.1.Liczmy idealne pozycje za pomocą algorytmu Tutte
13    for(int i = 0; i < graph->num_nodes; i++) {
14        tutte_iteration(graph,i, is_fixed, adj_list,new_pos, deg, center);
15    }
16
17
18    //4.2.Itercja po każdej parze nowych koordynat dla wierzchołków, jeśli one mają tę samą pozycję, to
19    ↪ rozsuwamy je
20    for (int a = 0;a<n; a++) {
21        for(int b =a+1;b<n;b++){
22            if(have_same_pos(new_pos[a], new_pos[b]) && is_fixed[a] !=1 && is_fixed[b]!=1){
23                offset(new_pos, a,b, difference);
24            }
25        }
26    }
27    //4.3.Sprawdzenie warunku pętli while oraz zapis policzonych pozycji
28    for(int i = 0; i < graph->num_nodes; i++) {
29        if(is_fixed[i] != 1) {
30            double change = distance(new_pos[i],graph->nodes[i].position);
31            if (change >max_change){
32                max_change = change;
33            }
34
35            //Zapis końcowych pozycji do Graph
36            graph->nodes[i].position.x = new_pos[i].x;
37            graph->nodes[i].position.y = new_pos[i].y;
38
39        }
40    }
41    //4.4.Zwolnienie zaalokowanej pamięci
42    free_adj_list(graph, adj_list);
43    free_deg(deg);
44
45
46    return EXIT_SUCCESS;
47 }

```

Główna pętla realizuje iteracyjne przybliżanie pozycji wierzchołków do ich stanów równowagi. Dodatkowo funkcja sprawdza kolizje (nakładające się pozycje) i stosuje losowe przesunięcie (`offset`) w celu ich rozdzielania. Pętla kończy się po osiągnięciu limitu iteracji lub gdy maksymalna zmiana pozycji wierzchołka spadnie poniżej progu `MINIMUM_CHANGE`.

12.6.2 Funkcja tutte_iteration

```
1 int tutte_iteration(Graph *graph, int iteration, int is_fixed[], uint** adj_list, Vector new_pos[], int
   ↪ deg[], Vector center){
2     double sum_x = 0.0;
3     double sum_y = 0.0;
4     // Jeśli wierzchołek należy do poligonu zewnętrznego, zachowujemy jego aktualną pozycję
5     if(is_fixed[iteration] == 1) {
6         new_pos[iteration] = graph->nodes[iteration].position;
7
8         return EXIT_SUCCESS;
9     }
10    // Specjalna obsługa wierzchołków mających tylko jednego sąsiada
11    if(deg[iteration]==1){
12        int neighbor = adj_list[iteration][0];
13        //Vektor w kierunku przeciwnym od środka
14        Vector oposite = subtract_vectors(graph->nodes[neighbor].position, center);
15
16        //Dzielimy przez dystancję między centrem a wierzchołkiem żeby otrzymać wektor
   ↪ kierunkowy z długością = 1
17        double len = distance(graph->nodes[neighbor].position, center);
18        if (len != 0) {
19            oposite.x /=len;
20            oposite.y /=len;
21        }
22        // Zabezpieczenie przed dzieleniem przez zero przy zbyt małym dystansie
23        if (len < EPSILON) return EXIT_SUCCESS;
24
25        //Mnożymy przez ustawioną outer_rad żeby wierzchołek okazał się na odległości outer_rad
   ↪ od sąsiada
26        double outer_rad = 10; // nie może być < margin
27        oposite = mult_by_num(oposite, outer_rad);
28
29        // Ustawiamy nową pozycję jako przesunięcie względem jednego sąsiada
30        new_pos[iteration].x = graph->nodes[neighbor].position.x + oposite.x;
31        new_pos[iteration].y = graph->nodes[neighbor].position.y + oposite.y;
32    }else{
33        //Dla wierzchołków o stopniu > 1 stosujemy zasadę środka ciężkości
34        for(int j = 0; j <deg[iteration]; j++) {
35            int neighbor = adj_list[iteration][j];
36            sum_x += graph->nodes[neighbor].position.x;
37            sum_y += graph->nodes[neighbor].position.y;
38        }
39        // Jeśli wierzchołek jest izolowany (stopień 0), przerywamy obliczenia
40        if (deg[iteration] == 0) return EXIT_SUCCESS;
41        // Nowa pozycja to średnia arytmetyczna położen wszystkich sąsiadów
42        new_pos[iteration].x = sum_x/deg[iteration];
43        new_pos[iteration].y = sum_y/deg[iteration];
44    }
45    }
46
47    return EXIT_SUCCESS;
48 }
49 }
```

Funkcja oblicza nową pozycję dla konkretnego wierzchołka. W przypadku wierzchołków o stopniu wyższym niż 1, stosowana jest średnia arytmetyczna pozycji sąsiadów. Wierzchołki o stopniu 1 są specjalnie odpychane od środka grafu.

Argumenty:

- graph - wskaźnik na graf.
- iteration - indeks wierzchołka.
- is_fixed - tablica wierzchołków stałych.
- adj_list - dynamiczna tablica dwuwymiarowa sąsiadów.

- new_pos - tablica, w której zapisywany jest wynik.
- deg - stopnie wierzchołków.
- center - geometryczny środek obszaru.

12.6.3 Funkcja have_same_pos

```
1 int have_same_pos(Vector a, Vector b){
2     return (fabs(a.x - b.x) < EPSILON && fabs(a.y - b.y) < EPSILON );
3 }
```

Funkcja porównuje dwa wektory pozycji. Zwraca prawdę (1), jeśli różnica między współrzędnymi jest mniejsza niż zdefiniowana stała EPSILON.

Argumenty:

- a, b - porównywane wektory pozycji.

12.6.4 Funkcja offset

```
1 void offset(Vector new_pos[], int iteration, int jiteration, double difference){
2     double angle = ((double)rand() / RAND_MAX) * 2.0 * M_PI;
3
4     double dx = cos(angle) * difference / 2.0;
5     double dy = sin(angle) * difference / 2.0;
6
7     new_pos[iteration].x += dx;
8     new_pos[iteration].y += dy;
9
10    new_pos[jiteration].x -= dx;
11    new_pos[jiteration].y -= dy;
12 }
```

Funkcja ta służy do zmiany pozycji dwóch wierzchołków, które znalazły się w tym samym punkcie przestrzeni. Rozsuwa je w przeciwnych kierunkach pod losowym kątem o dystans zdefiniowany przez parametr difference.

Argumenty:

- new_pos - tablica nowych pozycji.
- iteration, jiteration - indeksy kolidujących wierzchołków.
- difference - wartość przesunięcia.